

JADAVPUR UNIVERSITY
DEPARTMENT OF PHYSICS

Study of Molecular Dynamics
Simulation
Using Verlet Algorithm

$$\vec{F}_{i,j} = 24\epsilon \left[2 \left(\frac{\sigma}{r_{i,j}} \right)^{12} - \left(\frac{\sigma}{r_{i,j}} \right)^6 \right] \hat{r}_{i,j} \longrightarrow P(v) = \frac{v}{v_0^2} e^{-0.5 \left(\frac{v}{v_0} \right)^2}$$

"From Lennard-Jones dynamics to Maxwell-Boltzmann equilibrium."

Submitted by:
Mohak Das

Supervisor:
Prof. Debasish Lohar

Class: PG-I
Class Roll No: 002520702028
Exam Roll No: PPHD0261034

COMPUTATIONAL PHYSICS LAB

Abstract

This report presents a two-dimensional molecular dynamics simulation of argon atoms to investigate the emergence of macroscopic thermodynamic properties from classical Newtonian mechanics. The system was modeled using the Lennard-Jones interatomic potential, and the equations of motion were numerically integrated using the velocity Verlet algorithm in Fortran 90. To ensure physical validity and eliminate edge effects, periodic boundary conditions and a system of reduced units were employed. The simulation successfully demonstrated strict conservation of total energy over 100,000 time steps, validating the numerical stability and symplectic nature of the integration scheme. Furthermore, the steady-state velocities of the simulated particles were analyzed and found to be in excellent agreement with the theoretical 2D Maxwell-Boltzmann distribution. To explore finite-size effects, a comparative analysis was conducted between a baseline 25-particle system and an expanded 100-particle system under a constant number density. The results visually and statistically illustrate the thermodynamic limit, showcasing how larger microcanonical ensembles mitigate statistical fluctuations and seamlessly converge toward predictable macroscopic behavior.

Contents

1	Introduction	1
2	Theoretical Framework	1
2.1	The Physical Model: Lennard-Jones Potential	1
2.2	Time Integration: The Verlet Algorithm	3
2.2.1	The Standard Verlet Algorithm	3
2.2.2	The Velocity Verlet Algorithm	3
2.3	Simulation Environment Boundaries and Units	4
2.3.1	Periodic Boundary Conditions	4
2.3.2	Reduced Units	4
3	Numerical Implementation	4
3.1	The Global Parameter Module (<code>md_params</code>)	5
3.2	The Initialization Module (<code>md_init</code>)	5
3.3	The Forces Module (<code>md_forces</code>)	7
3.4	The Integrator Module (<code>md_integrator</code>)	8
3.5	The Energy Module (<code>md_energy</code>)	9
3.6	The Analysis Module (<code>md_analysis</code>)	10
3.7	The Main Program (<code>md_main</code>)	13
4	Results and Discussion	14
4.1	Spatial Evolution: Initial and Final States	14
4.2	Conservation of Energy	15
4.3	Velocity Distribution and the Thermodynamic Limit	16
5	Conclusion	17

1 Introduction

Molecular dynamics (MD) is a deterministic computational approach used to study the behavior of classical many-particle systems by simulating the motion of individual atoms or molecules. Unlike Monte Carlo simulations, which are designed to sample systems in thermal equilibrium with a heat bath, molecular dynamics focuses on tracking how an isolated system dynamically evolves in real time from one microscopic state to another. By applying Newton's equations of motion, we can determine how particle positions and velocities change over time, allowing us to bridge the gap between microscopic interactions and macroscopic thermodynamic observables. In this study, we simulate a two-dimensional gas of argon atoms. Because the typical kinetic energy of an argon atom at these scales is significantly lower than its excitation energy, atomic collisions do not alter the electronic configuration, fully justifying a classical mechanical treatment. The atoms interact exclusively through the Lennard-Jones potential, and their trajectories are integrated using the velocity Verlet algorithm. This specific integrator was chosen over the standard position Verlet method due to its explicit handling of velocities and superior numerical stability. While the baseline of this study focuses on a simplified toy model of 25 molecules, the scope of the simulation has been extended to include a 100-molecule system. By comparing these two systems while maintaining a strictly constant number density, this report aims to demonstrate not only the conservation of energy in an isolated system but also the profound impact of finite-size effects and the thermodynamic limit on the Maxwell-Boltzmann velocity distribution.

2 Theoretical Framework

2.1 The Physical Model: Lennard-Jones Potential

We are modelling a system of classical particles (like Argon atoms). For such noble gases, we can safely ignore internal electronic structures and treat them classically. The particles interact via the Lennard-Jones potential, a mathematically simple but physically robust model.

Let r_{ij} denote the separation between i th and j th atom, with $u(r_{ij})$ representing their interaction potential. The total potential energy of the system is given by:

$$U = \sum_{i=1}^{N-1} \sum_{j=i+1}^N u_{ij}$$

The interaction potential $u(r)$ between any two atoms separated by a distance r is defined as:

$$u(r) = 4\epsilon \left[\left(\frac{\sigma}{r} \right)^{12} - \left(\frac{\sigma}{r} \right)^6 \right] \quad (1)$$

The attractive term $\left(\frac{1}{r^6} \right)$ dominates at long distances due to weak Van der Waals force caused by electrostatic interactions between induced dipole moments. The repulsive term $\left(\frac{1}{r^{12}} \right)$ dominates at short distances due to quantum mechanical effects governed by Pauli exclusion principle, which prevents electron cloud overlap.

The parameter σ defines the characteristic length scale, and ϵ defines the depth of the potential well. The potential function is zero at $r = \sigma$ and rapidly approaches zero for $r > 2\sigma$. The minimum potential energy occurs at $r = 2^{\frac{1}{6}}\sigma$.

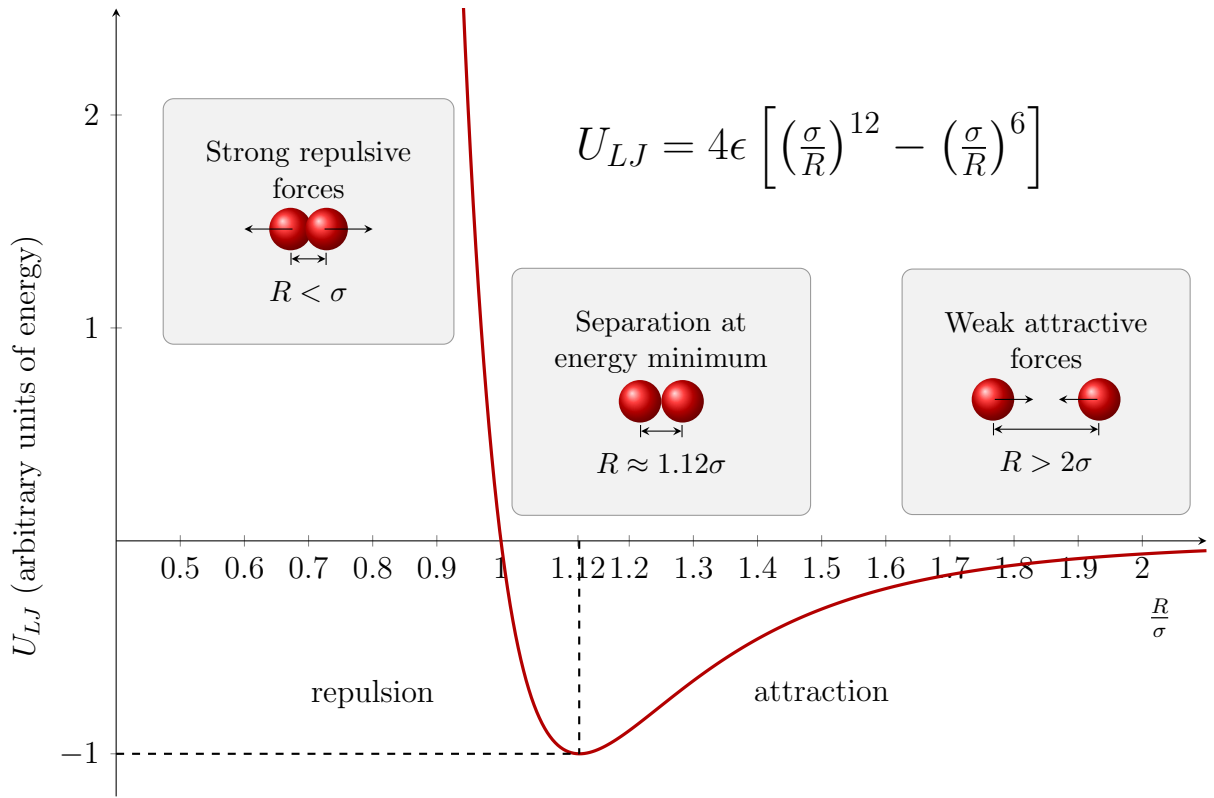


Figure 1: Lennard Jones Potential

The force exerted on i th atom by k th atom is given by the negative gradient of the interaction potential as follows,

$$\vec{f}_{k,i} = -\nabla_{k,i} u(r_{k,i}) = 24\epsilon \left[2 \left(\frac{\sigma}{r_{k,i}} \right)^{12} - \left(\frac{\sigma}{r_{k,i}} \right)^6 \right] \hat{r}_{i,j} \quad (2)$$

The x component is computed as,

$$\begin{aligned} F_{x,i} &= \sum_{k \neq i} f_{k,i} \cos \theta_{k,i} \\ &= \sum_{k \neq i} f_{k,i} \frac{x_i - x_k}{r_{k,i}} \\ &= 24\epsilon \sum_{k \neq i} \frac{x_i - x_k}{r_{k,i}^2} \left[2 \left(\frac{\sigma}{r_{k,i}} \right)^{12} - \left(\frac{\sigma}{r_{k,i}} \right)^6 \right] \end{aligned} \quad (3)$$

The y component is computed as,

$$\begin{aligned} F_{y,i} &= \sum_{k \neq i} f_{k,i} \sin \theta_{k,i} \\ &= \sum_{k \neq i} f_{k,i} \frac{y_i - y_k}{r_{k,i}} \\ &= 24\epsilon \sum_{k \neq i} \frac{y_i - y_k}{r_{k,i}^2} \left[2 \left(\frac{\sigma}{r_{k,i}} \right)^{12} - \left(\frac{\sigma}{r_{k,i}} \right)^6 \right] \end{aligned} \quad (4)$$

2.2 Time Integration: The Verlet Algorithm

To trace the trajectories, we must integrate Newton's second law, $m\ddot{x} = F(x)$, over discrete time steps h .

2.2.1 The Standard Verlet Algorithm

We can derive this via Taylor expansions of the position $x(t)$ moving forward and backward in time by h :

$$\begin{aligned} x(t+h) &= x(t) + h\dot{x}(t) + \frac{h^2}{2}\ddot{x}(t) + \frac{h^3}{6}x^{(3)}(t) + \mathcal{O}(h^4) \\ x(t-h) &= x(t) - h\dot{x}(t) + \frac{h^2}{2}\ddot{x}(t) - \frac{h^3}{6}x^{(3)}(t) + \mathcal{O}(h^4) \end{aligned}$$

Adding these two equations cleanly cancels out the odd-order velocity terms:

$$x(t+h) + x(t-h) = 2x(t) + h^2\ddot{x}(t) + \mathcal{O}(h^4)$$

The leading error term is $\mathcal{O}(h^4)$, indicating second-order accuracy. Substituting $\ddot{x}(t) = \frac{F(x(t))}{m}$, we get the explicit recursion relation:

$$x_{k+1} = 2x_k - x_{k-1} + \frac{h^2}{m}F(x_k)$$

While this is time-reversible and conserves energy well, it has a severe flaw: it does not explicitly calculate velocity. This makes tracking the system's kinetic energy and temperature cumbersome.

2.2.2 The Velocity Verlet Algorithm

To resolve this, we decouple the second-order differential equation into two first-order equations: $\dot{x}(t) = v(t)$ and $\dot{v}(t) = \frac{F(x(t))}{m}$. We calculate the explicit position update using the standard Taylor expansion:

$$x(t+h) = x(t) + hv(t) + \frac{h^2}{2m}F(x(t)) + \mathcal{O}(h^3) \quad (5)$$

For the velocity, we use the forces at both the current and the newly calculated next step to update the velocity explicitly:

$$\begin{aligned} v(t+h) &= v(t) + \frac{h}{m}F(x(t)) + \frac{h^2}{2m} \left[\frac{F(x(t+h)) - F(x(t))}{h} \right] + \mathcal{O}(h^3) \\ v(t+h) &= v(t) + \frac{h}{2m} [F(x(t+h)) + F(x(t))] + \mathcal{O}(h^3) \end{aligned} \quad (6)$$

Both the Verlet and Velocity Verlet algorithms are symplectic integrators, meaning they preserve phase-space volume and exhibit long-term stability. However, their stability and accuracy depend on the choice of h :

- A smaller h improves accuracy but increases computational cost.
- A larger h can introduce numerical instability, leading to energy drift.

Overall, the Velocity Verlet algorithm is favored in molecular dynamics due to its superior handling of both position and velocity while maintaining second-order accuracy and stability.

2.3 Simulation Environment Boundaries and Units

2.3.1 Periodic Boundary Conditions

Simulating a small number of particles (e.g., $N = 25$) in a rigid box introduces massive edge effects. We eliminate edges by wrapping the box into a torus topology. If a particle's coordinate exceeds the box length L_x , we apply $x = x - L_x$. For pair interactions, we use the *Minimum Image Convention*. This dictates that i th particle should only interact with the closest periodic image of j th particle. Let the raw displacement vector between i th and j th particle be:

$$\begin{aligned} dx &= x_i - x_j \\ dy &= y_i - y_j \end{aligned}$$

The maximum possible physical separation between two interacting particles in this mapped space is exactly half the box length in either direction ($\frac{L_x}{2}$ and $\frac{L_y}{2}$). If the absolute calculated distance exceeds this, it means an image of j th particle in an adjacent periodic cell is actually closer to i th particle than the primary j th particle itself.

We correct the displacement vector components systematically:

- If $dx > \frac{L_x}{2}$, j th particle is too far "left", so we interact with its image on the "right":
 $dx = dx - L_x$.
- If $dx < -\frac{L_x}{2}$, j th particle is too far "right", so we interact with its image on the "left":
 $dx = dx + L_x$.

The same logic applies to the y-components:

- If $dy > \frac{L_y}{2}$, then $dy = dy - L_y$.
- If $dy < -\frac{L_y}{2}$, then $dy = dy + L_y$.

2.3.2 Reduced Units

To avoid numerical underflow/overflow and make the code universally applicable, we non-dimensionalize the equations by setting $\sigma = 1$, $\epsilon = 1$, and $m = 1$. Consequently, time is measured in units of $\sigma\sqrt{m/\epsilon}$ (e.g., $\Delta t' = \frac{\Delta t}{\sigma\sqrt{m/\epsilon}}$).

3 Numerical Implementation

To build a clean, maintainable, and highly efficient Molecular Dynamics simulation, we constructed a modular architecture. Instead of single monolithic script, separating concerns into distinct modules ensures variables are safely scoped and physical routines are logically grouped.

The simulation is divided into the following modules:

1. **md_params:** A global data repository containing precision definitions, system parameters (particle count, box dimensions), and dynamically allocated arrays for kinematics (position, velocity, force).

2. **md_init:** This module handles the physical setup of the system, including placing particles on a spatial grid and assigning their initial thermodynamic states.
3. **md_forces:** This module is designed to compute forces due to Lennard-Jones interaction and enforce *Periodic Boundary Conditions*.
4. **md_integrator:** Advances the system forward in time using the *Velocity Verlet* algorithm.
5. **md_energy:** Calculates the kinetic energy, potential energy of the system when the system reach thermal equilibrium.
6. **md_analysis:** This module collects the velocities, generates the histogram, calculates v_0 from the average kinetic energy, and outputs the comparison data.
7. **md_main:** The driver program that links the modules, executes the main time loop, and outputs data for visualization.

3.1 The Global Parameter Module (md_params)

First, we establish a robust data structure. Using double precision is critical in MD to prevent floating-point drift, especially when calculating the r^{-12} repulsive term of the Lennard-Jones potential.

```

1 module md_params
2     implicit none
3     integer, parameter :: dp = selected_real_kind(15, 307)
4
5     integer :: Nmol, Nstep
6     real(dp) :: dt, Lx, Ly
7
8     real(dp), allocatable :: x(:), y(:)
9     real(dp), allocatable :: vx(:), vy(:)
10    real(dp), allocatable :: Fx(:), Fy(:)
11    real(dp), allocatable :: Fx1(:), Fy1(:)
12 contains
13
14 end module md_params

```

3.2 The Initialization Module (md_init)

To avoid infinite overlaps, we place the molecules in an $N_x \times N_y$ grid. To prevent the system from being perfectly crystalline, we introduce a small random spatial perturbation to each coordinate.

For a particle p mapped to grid indices (i, j) , the initial positions are:

$$x_p = \frac{L_x}{N_x}(i - 0.25 \times \text{rand})$$

$$y_p = \frac{L_y}{N_y}(j - 0.25 \times \text{rand})$$

where rand is a uniform random number in the interval $[0, 1)$.

We assign initial velocities drawn from a uniform random distribution ranging from $[-0.5v_{initial}, 0.5v_{initial}]$. If we assign velocities purely randomly, the net momentum of the system will likely be non-zero, causing the entire container of gas to drift. To prevent this, we must ensure the Center of Mass (COM) velocity is strictly zero.

Let the total momentum be $\vec{P} = \sum_{i=1}^{N_{mol}} m_i \vec{v}_i$. Because we are operating in reduced units where $m_i = 1$, the COM velocity is simply the arithmetic mean of the individual velocities:

$$\vec{V}_{COM} = \frac{1}{N_{mol}} \sum_{i=1}^{N_{mol}} \vec{v}_i$$

To eliminate drift, we apply a Galilean transformation, shifting the reference frame to the center of mass by subtracting $\vec{V}_{COM}$ from every particle's velocity:

$$\vec{v}'_i = \vec{v}_i - \vec{V}_{COM}$$

```

1 module md_init
2   use md_params
3   implicit none
4
5 contains
6   subroutine init_system(Nx, Ny, v_initial)
7     integer, intent(in) :: Nx, Ny
8     real(dp), intent(in) :: v_initial
9
10    integer :: i, j, p
11    real(dp) :: rnd, vx_tot, vy_tot
12
13    allocate(x(Nmol), y(Nmol), vx(Nmol), vy(Nmol))
14    allocate(Fx(Nmol), Fy(Nmol), Fx1(Nmol), Fy1(Nmol))
15
16    call random_seed()
17
18    p = 0
19    do i = 1, Nx
20      do j = 1, Ny
21        p = p + 1
22        if (p > Nmol) exit
23
24        call random_number(rnd)
25        x(p) = (Lx/real(Nx,dp))*(i-0.25_dp*rnd)
26
27        call random_number(rnd)
28        y(p) = (Ly/real(Ny,dp))*(j-0.25_dp*rnd)
29      end do
30    end do
31
32    vx_tot = 0.0_dp
33    vy_tot = 0.0_dp
34
35    do i = 1, Nmol
36      call random_number(rnd)
37      vx(i) = v_initial*(rnd-0.5_dp)
38
39      call random_number(rnd)

```

```

40      vy(i) = v_initial*(rnd-0.5_dp)
41
42      vx_tot = vx_tot + vx(i)
43      vy_tot = vy_tot + vy(i)
44  end do
45  vx_tot = vx_tot/real(Nmol, dp)
46  vy_tot = vy_tot/real(Nmol, dp)
47
48  do i = 1, Nmol
49      vx(i) = vx(i) - vx_tot
50      vy(i) = vy(i) - vy_tot
51  end do
52  end subroutine init_system
53 end module md_init

```

3.3 The Forces Module (md_forces)

To calculate forces, we first calculate the displacement between i th and j th particle using *Minimum Image Convention* we discussed earlier. Once we have the minimum image displacement (dx, dy) , the squared distance is simply $r^2 = dx^2 + dy^2$. The force calculation will be according to equation (3) and (4).

Computationally, calculating high-order powers (like r^{12}) or square roots is extraordinarily expensive. We can optimize this rigorously by working entirely in squared inverse terms. Because we are using reduced units where $\sigma = 1$ and $\epsilon = 1$, the calculation collapses beautifully.

Let us define an intermediate variable for the inverse squared distance:

$$t_1 = \frac{1}{r^2}$$

We can obtain the higher order terms through simple multiplication:

$$t_2 = (t_1)^3 = \frac{1}{r^6}$$

The magnitude of the force vector becomes:

$$F_{scalar} = 24 \cdot t_1 \cdot (2t_2^2 - t_2)$$

The force components are then simply $F_x = dx \cdot F_{scalar}$ and $F_y = dy \cdot F_{scalar}$.

By Newton's Third Law, the force particle i exerts on j is equal and opposite to the force j exerts on i ($\vec{F}_{ji} = -\vec{F}_{ij}$). We can cut our computational load in half by looping i from 1 to $N_{mol} - 1$ and j from $i + 1$ to N_{mol} , applying the computed force to i and its negative to j simultaneously.

```

1 module md_forces
2   use md_params
3   implicit none
4 contains
5
6   subroutine calculate_forces()
7     integer :: i, j
8     real(dp) :: dx, dy, r2, t1, t2, f_scalar

```

```

9
10     Fx = 0.0_dp
11     Fy = 0.0_dp
12
13     do i = 1, Nmol-1
14         do j = i+1, Nmol
15             dx = x(i) - x(j)
16             dy = y(i) - y(j)
17
18             if ( dx > Lx/2.0_dp ) then
19                 dx = dx - Lx
20             else if ( dx < -Lx/2.0_dp ) then
21                 dx = dx + Lx
22             end if
23
24             if ( dy > Ly/2.0_dp ) then
25                 dy = dy - Ly
26             else if ( dy < -Ly/2.0_dp ) then
27                 dy = dy + Ly
28             end if
29
30             r2 = dx**2 + dy**2
31             t1 = 1.0_dp/r2
32             t2 = t1**3
33             f_scalar = 24.0_dp * t1 * (2.0_dp*t2**2-t2)
34
35             Fx(i) = Fx(i) + dx * f_scalar
36             Fy(i) = Fy(i) + dy * f_scalar
37
38             Fx(j) = Fx(j) - dx * f_scalar
39             Fy(j) = Fy(j) - dy * f_scalar
40         end do
41     end do
42     end subroutine calculate_forces
43 end module md_forces

```

3.4 The Integrator Module (md_integrator)

To advance the system forward in time, we implement the *Velocity Verlet* algorithm as discussed in equation (5) and (6). To calculate the new velocity $v(t+h)$, we strictly require the new force $F(x(t+h))$. Therefore, the order of operations in our code must be precisely sequenced:

1. **Advance Positions:** Calculate $x(t+h)$ and $y(t+h)$ using current velocities and current forces.
2. **Enforce Boundaries:** Immediately apply *Periodic Boundary Conditions* to the new positions to ensure particles don't escape the simulation box.
3. **Store Old Forces:** Save the current forces ($F(x(t))$) into temporary arrays ($Fx1$, $Fy1$) because we are about to overwrite them.
4. **Update Forces:** Call `calculate_forces()` subroutine to compute the new forces $F(x(t+h))$ based on the newly updated positions.

5. **Advance Velocities:** Calculate $v(t + h)$ using the stored old forces and the newly computed forces.

```

1 module md_integrator
2   use md_params
3   use md_forces
4   implicit none
5
6 contains
7   subroutine integrate_step()
8     integer :: i
9
10    do i = 1, Nmol
11      x(i) = x(i) + dt*vx(i) + 0.5_dp * Fx(i)*dt**2
12      y(i) = y(i) + dt*vy(i) + 0.5_dp * Fy(i)*dt**2
13
14      if ( x(i) > Lx ) then
15        x(i) = x(i) - Lx
16      else if ( x(i) < 0.0_dp ) then
17        x(i) = x(i) + Lx
18      end if
19
20      if ( y(i) > Ly ) then
21        y(i) = y(i) - Ly
22      else if ( y(i) < 0.0_dp ) then
23        y(i) = y(i) + Ly
24      end if
25    end do
26
27    do i = 1, Nmol
28      Fx1(i) = Fx(i)
29      Fy1(i) = Fy(i)
30    end do
31
32    call calculate_forces()
33
34    do i = 1, Nmol
35      vx(i) = vx(i) + 0.5_dp * dt * (Fx(i) + Fx1(i))
36      vy(i) = vy(i) + 0.5_dp * dt * (Fy(i) + Fy1(i))
37    end do
38  end subroutine integrate_step
39 end module md_integrator

```

3.5 The Energy Module (md_energy)

This module is created specifically for tracking the thermodynamics of the system. While calculating forces and potential energy in the same loop is computationally faster, keeping them separate increase the maintainability.

```

1 module md_energy
2   use md_params
3   implicit none
4
5 contains
6   subroutine calculate_potential(pot)

```

```

7      real(dp), intent(out) :: pot
8      integer :: i, j
9      real(dp) :: dx, dy, r2, t1, t2
10
11     pot = 0.0_dp
12
13     do i = 1, Nmol-1
14         do j = i+1, Nmol
15             dx = x(i) - x(j)
16             dy = y(i) - y(j)
17
18             if ( dx > Lx/2.0_dp ) then
19                 dx = dx - Lx
20             else if ( dx < -Lx/2.0_dp ) then
21                 dx = dx + Lx
22             end if
23
24             if ( dy > Ly/2.0_dp ) then
25                 dy = dy - Ly
26             else if ( dy < -Ly/2.0_dp ) then
27                 dy = dy + Ly
28             end if
29
30             r2 = dx**2 + dy**2
31             t1 = 1.0_dp/r2
32             t2 = t1**3
33
34             pot = pot + 4.0_dp *(t2**2-t2)
35         end do
36     end do
37 end subroutine calculate_potential
38
39 subroutine calculate_kinetic(kinetic)
40     real(dp), intent(out) :: kinetic
41     integer :: i
42     real(dp) :: v_sq
43
44     kinetic = 0.0_dp
45     do i = 1, Nmol
46         v_sq = vx(i)**2 + vy(i)**2
47         kinetic = kinetic + 0.5_dp * v_sq
48     end do
49 end subroutine calculate_kinetic
50 end module md_energy

```

3.6 The Analysis Module (md_analysis)

To verify the simulation is physically accurate, we must prove that the steady-state velocities of the argon atoms obey the Maxwell-Boltzmann distribution.

The exact 2D Maxwell-Boltzmann speed distribution is:

$$P(v) = \frac{m}{k_B T} v e^{-\frac{mv^2}{2k_B T}} \quad (7)$$

In the simulation's reduced units, mass $m = 1$. By the Equipartition Theorem in 2D, the average kinetic energy per particle is $\langle KE \rangle = \frac{2}{2} k_B T = k_B T$. Therefore, the most probable

velocity v_0 satisfies $v_0^2 = k_B T$. Substituting this to equation (7) yields:

$$P(v) = \frac{v}{v_0^2} e^{-0.5\left(\frac{v}{v_0}\right)^2} \quad (8)$$

To compare the discrete simulation data against this continuous theoretical curve, we must construct a normalized histogram:

1. **Equilibration:** The system initially exhibits transient, non-physical behavior as the grid arrangement breaks down. We only sample velocities after a designated equilibrium time (e.g., $t > 10000$ steps).
2. **Binning:** We divide the range of observed speeds into discrete bins of width h . We count how many times a particle's speed falls into each bin.
3. **Normalization:** The raw counts must be divided by the total number of samples and the bin width h to represent a probability density function. A highly accurate way to find the total integration area of the discrete data is using Simpson's 1/3 Rule.

```

1 module md_analysis
2   use md_params
3   implicit none
4
5 contains
6   subroutine velocity_distribution(vel_list, Nstep_total, bin_no,
7     filename)
8     integer, intent(in) :: bin_no, Nstep_total
9     real(dp), intent(in) :: vel_list(Nmol, Nstep_total)
10    character(len=*), intent(in) :: filename
11
12    integer :: i, j, k, count, index_max
13    real(dp) :: max_val, min_val, h, s, integration, v
14    real(dp) :: KE_avg, KE_tot, v_most, fn_max
15    real(dp), allocatable :: fn(:)
16
17    allocate(fn(bin_no))
18    fn = 0.0_dp
19
20    max_val = maxval(vel_list)
21    min_val = 0.0_dp
22    h = (max_val - min_val) / real(bin_no, dp)
23
24    s = min_val
25    do k = 1, bin_no
26      count = 0
27      do j = 10001, Nstep_total
28        do i = 1, Nmol
29          if (vel_list(i,j) >= s .and. vel_list(i,j) < (s+h))
30            count = count + 1
31          end if
32        end do
33      end do
34      fn(k) = real(count, dp)
35      s = s+h
36    end do
37  end subroutine

```

```

35     end do
36
37     call simpson(fn, bin_no, h, integration)
38
39     fn_max = fn(1)
40     index_max = 1
41     do i = 2, bin_no
42         if ( fn(i) > fn_max ) then
43             fn_max = fn(i)
44             index_max = i
45         end if
46     end do
47     v_most = min_val + real(index_max, dp)*h
48
49     KE_tot = 0.0_dp
50     do j = 10001, Nstep_total
51         do i = 1, Nmol
52             KE_tot = KE_tot + 0.5*vel_list(i,j)**2
53         end do
54     end do
55     KE_avg = KE_tot/real((Nstep_total - 10000)*Nmol, dp)
56
57     open(1004, file=filename, status="replace")
58     do i = 1, bin_no
59         v = min_val + (real(i, dp) - 0.5_dp)*h
60         write(1004, *) v, fn(i)/integration, P_theoretical(v, sqrt(
KE_avg)), P_theoretical(v, v_most)
61     end do
62     close(1004)
63
64     write(*, '(A, G0.10)') "Simulation V_most: ", v_most
65     write(*, '(A, G0.10)') "Theoretical V_most: ", sqrt(KE_avg)
66
67     deallocate(fn)
68 end subroutine velocity_distribution
69
70 subroutine simpson(fn, bin_no, h, integration)
71     real(dp), intent(in) :: fn(:), h
72     integer, intent(in) :: bin_no
73     real(dp), intent(out) :: integration
74     real(dp) :: s1, s2
75     integer :: i, j
76
77     s1 = 0.0_dp
78     s2 = 0.0_dp
79
80     do i = 2, bin_no - 1, 2
81         s1 = s1 + fn(i)
82     end do
83
84     do j = 3, bin_no - 2, 2
85         s2 = s2 + fn(j)
86     end do
87
88     integration = (h/3.0_dp) * (fn(1) + fn(bin_no) + 4.0_dp*s1 + 2.0
_dp*s2)

```

```

89     end subroutine simpson
90
91     function P_theoretical(v, v0) result(P)
92         real(dp), intent(in) :: v, v0
93         real(dp) :: P
94         P = (v/v0**2) * exp(-0.5_dp*(v/v0)**2)
95     end function P_theoretical
96 end module md_analysis

```

3.7 The Main Program (md_main)

Since all the modules are defined above, we can write the main driver. This program will use all the modules, initialize the system, run the *Velocity Verlet* time loop, and output the energy data to track the conservation of energy along with the velocity distribution analysis.

```

1 program md_main
2     use md_params
3     use md_init
4     use md_forces
5     use md_integrator
6     use md_energy
7     use md_analysis
8     implicit none
9
10    integer :: step, i, Nx, Ny
11    real(dp) :: KE, PE, TE
12    real(dp) :: v_initial
13    real(dp), allocatable :: v_history(:, :)
14    character(len=256) :: velocity_dat
15
16    velocity_dat = "data_25/Velocity_dist.dat"
17    open(1000, file="data_25/Initial.dat", status="replace")
18    open(1001, file="data_25/Final.dat", status="replace")
19    open(1002, file="data_25/Energy.dat", status="replace")
20
21    Nmol = 25
22    Nstep = 100000
23    Nx = ceiling(sqrt(real(Nmol, dp)))
24    Ny = ceiling(real(Nmol, dp)/real(Nx, dp))
25    dt = 0.01_dp
26    Lx = 4.0_dp*sqrt(real(Nmol, dp))
27    Ly = 4.0_dp*sqrt(real(Nmol, dp))
28    v_initial = 5.0_dp
29
30    call init_system(Nx, Ny, v_initial)
31    allocate(v_history(Nmol, Nstep))
32
33    do i = 1, Nmol
34        write(1000, *) x(i), y(i), vx(i), vy(i)
35    end do
36
37    call calculate_forces()
38
39    do step = 1, Nstep
40        call integrate_step()
41    end do

```



```

42     do i = 1, Nmol
43         v_history(i, step) = sqrt(vx(i)**2 + vy(i)**2)
44     end do
45
46     if ( step > 10000 ) then
47         call calculate_kinetic(KE)
48         call calculate_potential(PE)
49         TE = KE + PE
50         write(1002, *) step, KE, PE, TE
51     end if
52 end do
53
54 do i = 1, Nmol
55     write(1001, *) x(i), y(i), vx(i), vy(i)
56 end do
57
58 write(*, '(A)') "Simulation completed successfully."
59 close(1000)
60 close(1001)
61 close(1002)
62
63 write(*, '(A)') "Analyzing velocity distribution..."
64 call velocity_distribution(v_history, Nstep, 1000, velocity_dat)
65 end program md_main

```

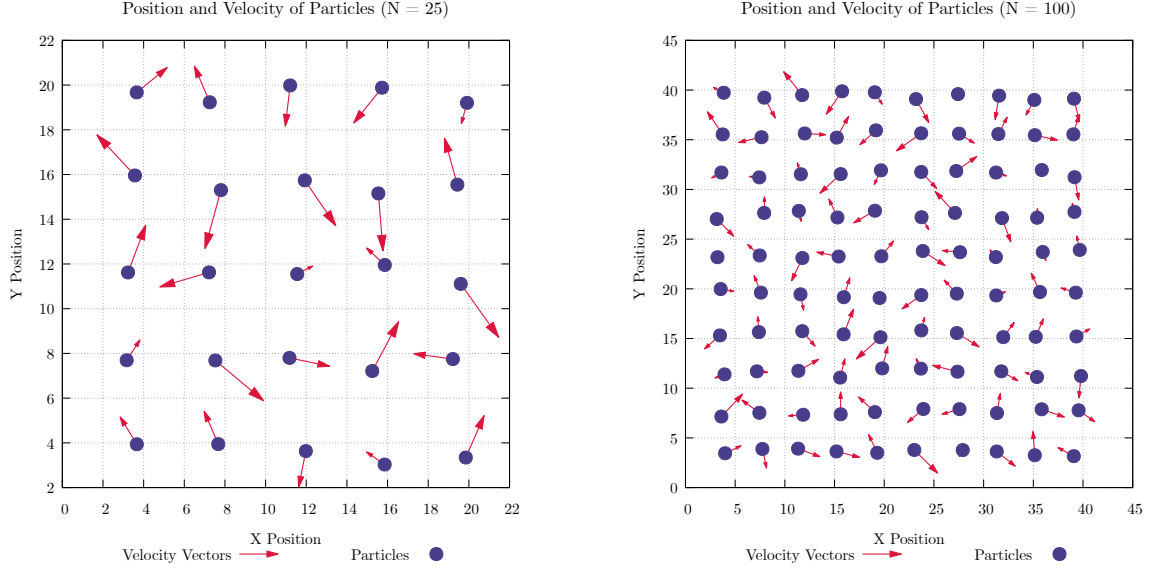
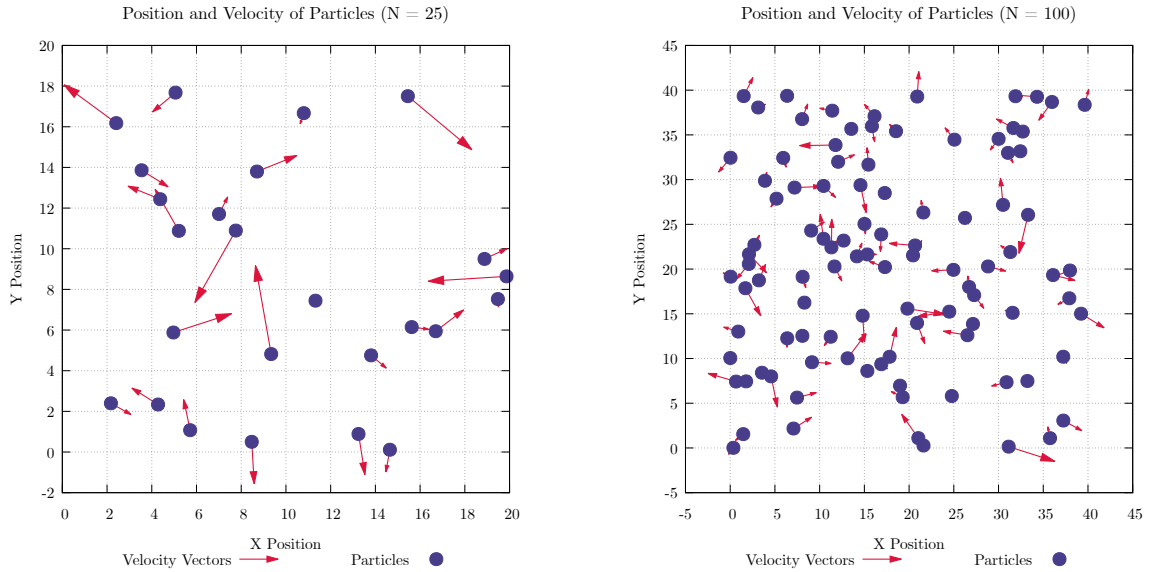
4 Results and Discussion

The molecular dynamics simulation was executed over $N_{step} = 100,000$ time steps with a reduced time step of $\Delta t = 0.01$. To investigate finite-size effects and the emergence of macroscopic thermodynamics, the simulation was performed for two distinct system sizes: a baseline model of $N = 25$ atoms in a 20×20 box, and an expanded model of $N = 100$ atoms in a 40×40 box. The box dimensions were dynamically scaled to ensure the number density ($\rho = 0.0625$) remained strictly constant across both simulations.

4.1 Spatial Evolution: Initial and Final States

At the start of the simulation ($t = 0$), particles were placed in a perfect grid (5×5 for $N = 25$, and 10×10 for $N = 100$) to prevent infinite repulsive forces from overlapping coordinates, and were assigned random initial velocities.

By the end of the simulation ($t = 100,000$), the spatial distribution of the particles had completely randomized. The initial ordered grid structure "melted" as the systems evolved into chaotic, gas-like states. The comparative final-state quiver plots visually highlight the scale difference between the two systems. While the 25-particle system exhibits large empty voids characteristic of a highly confined microstate, the 100-particle system demonstrates a much more uniform spatial distribution. This visual evidence reinforces that the larger system behaves more like a continuous thermodynamic fluid, properly representing a well-mixed gas.

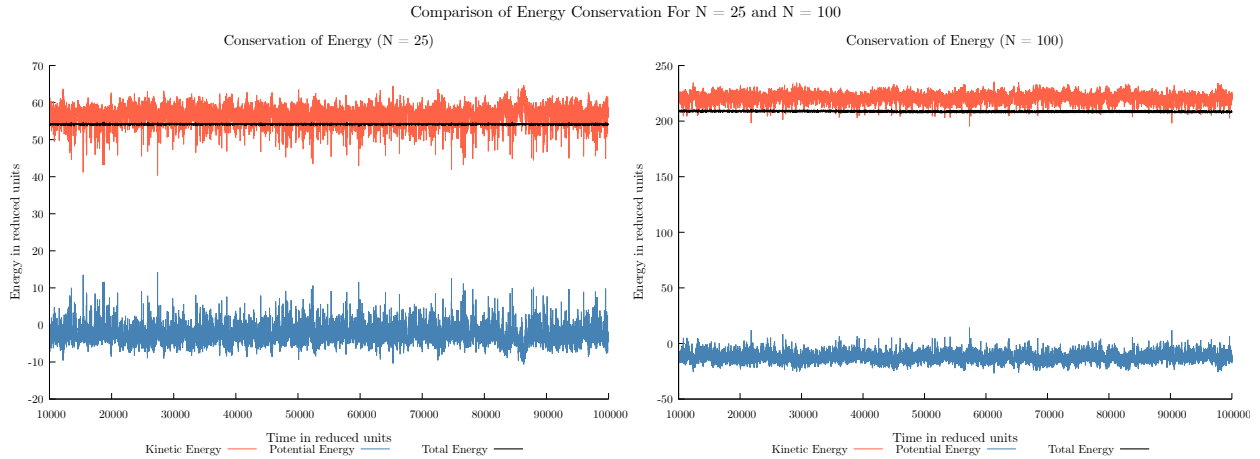
Comparison of Initial State for $N = 25$ and $N = 100$ Comparison of Final State for $N = 25$ and $N = 100$ 

4.2 Conservation of Energy

Because the simulated system is strictly isolated, classical mechanics dictates that the total energy must remain constant. The energy time-series plots for both $N = 25$ and $N = 100$ confirm this fundamental principle, displaying a perfectly flat horizontal line for the total energy throughout the entire 100,000-step integration.

While the total energy remained constant, the kinetic and potential energies exhibited continuous, dynamic fluctuations. The data reveals a perfect counter-balancing relationship: every localized increase in kinetic energy is met with an identical decrease in potential energy (and vice versa).

Furthermore, comparing the two systems demonstrates that energy is an extensive thermodynamic property. The total energy of the $N = 25$ system stabilized at approximately 55 reduced units, whereas the $N = 100$ system stabilized at roughly 220 reduced units. Quadrupling the number of particles directly quadrupled the total energy, perfectly aligning with theoretical expectations.

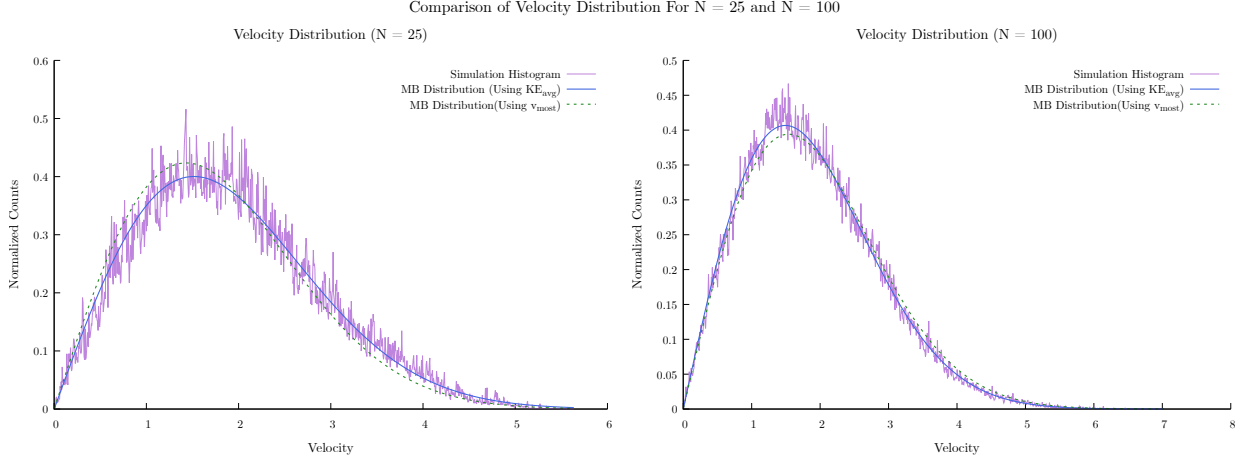


4.3 Velocity Distribution and the Thermodynamic Limit

To verify that the systems reached thermal equilibrium, the simulated velocities were sampled and compiled into histograms using 1,000 bins. These were plotted against the continuous theoretical 2D Maxwell-Boltzmann distribution, calculated using both the most probable velocity (V_{most}) and the root-mean-square velocity derived from the average kinetic energy ($\sqrt{KE_{avg}}$).

The side-by-side comparison of the velocity distributions provides a striking demonstration of the **thermodynamic limit**.

- For the $N = 25$ system, the velocity histogram is highly jagged and noisy. Because it is a small microcanonical ensemble, it is highly susceptible to statistical fluctuations, fitting only loosely around the theoretical Maxwell-Boltzmann curve.
- For the $N = 100$ system, the law of large numbers heavily mitigates these statistical anomalies. The expanded system's histogram fills in the gaps and smoothly hugs the theoretical probability density function.



This statistical convergence proves that while small numerical models can simulate basic Newtonian trajectories, larger particle counts are required to suppress microscopic fluctuations and accurately extract macroscopic thermodynamic observables.

5 Conclusion

This study successfully implemented a modular molecular dynamics simulation in Fortran 90, utilizing the velocity Verlet algorithm to accurately model the motion of argon atoms under the Lennard-Jones potential. The numerical results unequivocally validated the core physical principles governing isolated classical systems. Most notably, the total energy of the system remained strictly conserved over 100,000 time steps. The continuous fluctuations in kinetic and potential energy exhibited a perfect counter-balancing relationship, ensuring thermodynamic stability throughout the integration process. Furthermore, by extracting macroscopic observables from the microscopic atomic trajectories, the simulation successfully reproduced the theoretical 2D Maxwell-Boltzmann velocity distribution. The extension of the simulation from $N = 25$ to $N = 100$ particles provided a compelling visual and statistical demonstration of the thermodynamic limit. While the 25-atom system exhibited the expected statistical noise of a small microcanonical ensemble, the 100-atom system drastically reduced these fluctuations, adhering tightly to the theoretical probability density function and visually representing a continuous, well-mixed gas. Overall, this computational model effectively captured the microscopic interactions of atomic particles, proving the reliability of the velocity Verlet method and demonstrating how macroscopic thermodynamic equilibrium emerges directly from deterministic Newtonian mechanics.